

Spring 2026

Introduction to Artificial Intelligence

Project 2: RAG Parameter Exploration

Due Date: 2026/6/12 (Fri.) 23:59 PM

Introduction

Retrieval-Augmented Generation (RAG) is a framework that enhances Large Language Models (LLMs) by incorporating external knowledge through retrieval mechanisms.

In this assignment, you will:

1. Explore how different RAG configurations affect model performance.
 2. Build a practical RAG-based application for a meeting assistant system.
-

Provided Materials

-  Colab sample code ([link](#))
 -  RAG Judge System ([link](#))
 -  Report template ([link](#))
 -  Notion QA page ([link](#))
 -  Introduction video ([link](#))
-

RAG Parameter Exploration

Objective

Understand how RAG improves over pure LLM by experimenting with different configurations.

Environment Setup

- You can only run the code on Colab.

- Instead of editing the sample code directly from the provided Colab link, **please make a copy first. Otherwise, your changes may not be saved properly.**
 - **Colab has free usage limits, so start your assignment early to avoid running out of credits.**
 - You **are allowed** to use external libraries unless otherwise specified.
-

Model Configuration

To ensure fairness and consistency in grading for this AI assignment, your Large Language Model (LLM) decoding parameters must strike a balance between predictability and output quality.

We will provide the following parameter settings in the given code.

Please do not change the variables below; any changes of these variables will result in a score of 0.

Please adhere to the following technical specifications when configuring your model:

1. Model Name

In this assignment, we will use a local open-source model. This part of the assignment will use the same model. Please do not fine-tune the model.

- Setting: **unsloth/gemma-3-4b-it**

2. Temperature

Temperature controls the "smoothness" of the probability distribution. It directly dictates the likelihood of the model selecting tokens outside the highest-probability candidate.

- Setting: **None**

3. Do Sample

Top-K sampling filters the vocabulary to include only the top K most likely next tokens, redistributing the probability mass among them.

- Setting: **False**
-

Tasks

0. Task levels:

- Simple: LLM only

- Medium: BM25 + LLM
 - Strong: embedding + LLM
 - Hybrid: LLM+BM25+embedding
-

1. Simple Baseline: LLM Only

Use only a Large Language Model to answer queries **without any retrieval**.

Requirements

- Directly input the query into the LLM
- No external knowledge base is allowed

Goal

- Serve as a baseline to observe hallucination and knowledge limitations
- Try using **Prompt Engineering** to get better performance without using any retrieval

Questions to answer

- Explain what kind of Prompt Engineering you try and describe your observation.
- **Provide examples** to demonstrate what kind of queries get performance improvement through prompt engineering.

Sources

Here are some sources for you to know more about prompt engineering:

- [Prompt Engineering Guide](#)
 - [A SURVEY OF PROMPT ENGINEERING METHODS IN LARGE LANGUAGE MODELS FOR DIFFERENT NLP TASKS](#)[Prompt Engineering Guide](#)
 - [Prompt engineering overview - Claude API Docs](#)
 - [Prompt engineering | OpenAI API](#)
-

2. Medium: BM25 + LLM

Implement a basic RAG pipeline using **BM25 (sparse retrieval)**.

Requirements

Your pipeline must include:

- Document chunking

- BM25 retrieval (no embeddings)
- LLM generation

Notes

- BM25 retrieval is based on keyword matching
- Changing k_1 and b **requires rebuilding** the BM25 model
- No embedding model should be used in this part

Goal

- Try to change **k_1 value** and find out what it is representing
- Try to change **b value** and find out what it is representing

Questions to answer

- Briefly describe what happened after adding retrieval into the system.
- What does **k_1 value** and **b value** represent?

Sources

Here are some sources for you to know more about BM25:

- [Practical BM25 - Part 2: The BM25 Algorithm and its Variables | Elastic Blog](#)
 - [What is BM25 \(Best Matching 25\) Algorithm - GeeksforGeeks](#)
-

3. Strong: embedding + LLM

Requirements

You must experiment with **all** of the following:

- Prompt design: How retrieved context is formatted and presented to the LLM
- Chunk size: Length of each document chunk, semantic segmentation.
- Chunk overlap: Overlapping tokens between adjacent chunks

You have to implement the UMAP plotting part. **Please refer to the libraries provided in the sources to complete this.**

Goal

- Try to change **prompt design** and observe the changes.
- Using chunk size=512, compare the most relevant 10 chunks between BM25 and embedding.
- Try to adjust chunk size and chunk overlap and observe the changes.

Questions to answer

- Show your final prompt design, and explain your strategy.
- Using chunk size=512, compare the most relevant 10 chunks between BM25 and embedding in the semantic space, apply UMAP for dimensionality reduction to compress the chunk vectors into 3D. **Provide a screenshot** of plotting the retrieved chunks in a 3D space and color-coded them to visualize the differences between BM25 and embedding. (You must use colab to plot it) And describe your observation. You can use any query to finish this part.
- Please experience different chunk sizes, chunk overlaps to observe the influence of the results. Fill in the table supplied in the report template.

Source

- [UMAP](#)
 - [understanding-umap](#)
 - [Plotly Python Graphing Library](#)
-

4. Hybrid: LLM+BM25+embedding

Requirements

In task 4, we will not restrict the methods you use, **but the parameters (LLM: model name, top-k, temperature) must not be changed, and manual retrieval is strictly prohibited**. If any methods mentioned above are used, the scores for **task 4 and the implementation scores will lead to 0**.

Goal

- Get a score on the system leaderboard as high as possible.
- Describe your strategy to get the high score on the system.

Questions to answer

- Describe how you integrate the results from BM25 and embedding-based retrieval.
 - Provide at least ONE example query from sample_query.jsonl to demonstrate how your hybrid retrieval improves performance.
 - Fill in the table supplied in the report template to show your setting in this phase.
 - Describe any additional techniques (beside example code) you applied to further improve performance.
-

Evaluation

- Submit your prediction file to the **judge system**, **otherwise you will not receive any score** for the implementation part.
 - Every group has **3 submission quota per day**(evaluation is costly), reset at **08:00AM (GMT+8)**
 - Please note that the submission has the following limitations: File max 5 MB, Per-question prompt max 131072 chars, answer max 1024 chars.
 - Evaluation time per prediction file is about **10** minutes if there are not too many students submitting at the same time
 - You can check the scoring rubrics at the [leaderboard](#) tab.
 - **You must use the NYCU IP address to access the system. If you are not in school, please use VPN.**
 - **If a Prompt Injection is detected, the ranking score will be 0.**
 - **You should NOT modify your prediction files manually, otherwise the score will be 0.**
-

Upload Format

Please follow the submission layout and submit the three files separately:

None

```
{515512|515513}_{groupNum}_HW4_report.pdf  
{515512|515513}_{groupNum}_HW4_code.ipynb  
{515512|515513}_{groupNum}_HW4_result.json
```

example: 515512_Group01_HW4_report.pdf

Grading

Total (100%)

- **Report (70%)**
 - Task1 (6%)
 - Task2 (14%)
 - Task3 (20%)
 - Task4 (30%)
- **Implementation (30%)**
 - Pass Baseline: Get a public score over 13.2 on the system (10%)
 - System Ranking (20%)
 - We will use this formula to calculate your ranking score:

- $\text{score} = 20 * [1 - (\text{your ranking number} / \text{total team amount})]$
- Note that the ranking displayed on the system is based **only on public test cases**. However, the final score will use the ranking including **private test cases**.
- **Peer Evaluation**